Preprint 1–9, 2026

TLTS-Compilation: A Neurosymbolic Framework for Type-Safe and Verifiable Transformers

Mike H Lee MIKE@OASIS-X.IO
O-X

Erratum (2026-10-21). The (B′) soundness numbers in the first version of this paper were a single draw that depended on Python's string-hash seed (`PYTHONHASHSEED`), not only on the numpy seed. With sorted label sets, over 10 numpy seeds (42–51), (B′) reaches 63.3% (range 60.8–65.1%) under GOOD (was 61.5%) and 4.5% under BAD (was 4.3%); on the cyclic TLTS it reaches 6.6% (was 8.0%). The findings do not change: reachability masking helps a well-calibrated model by about 15 points, does nothing for a mis-calibrated one, and collapses to no enforcement on a cyclic ontology; (C) and (D) stay at 100%. Variants A, C and D were not affected at the reported precision. The other (B′) columns of Table 2 and the (B′) column of Table 4 are single draws that were not re-run. Details: `AUDIT_2026-10-21.md`.

Abstract

Verifying that a language model stayed within the rules of a structured domain—that it did not invent an absent relation or skip a required step—is currently faith-based. We frame the problem as compiling a typed labeled transition system (TLTS) into a transformer, and show that two recent threads—typed attention and program-compiled transformers—are the same construction, differing only in whether the transition relation is deterministic. The framework names three points in the inference pipeline where the rule can be enforced (in-FFN gates, pre-decoder logit masks, post-hoc audit), each with a distinct cost profile. A synthetic harness confirms its predictions, including a non-obvious one: attention-layer reachability masking is not enough for trajectory soundness; the decoder must also check direct-edge admissibility. We deliver a JSON audit-certificate format that a third party can re-check without model weights, and argue it should become a publication norm for compiled-transformer work. Trained-model evaluation is future work.

1. Introduction

A deployed language model is asked to generate inside a structured domain—an ontology, a protocol, a program semantics. Today, nothing in the forward pass forces it to stay there: any token may attend to any other, and any next-token may be sampled, regardless of whether the implied move is admissible in the domain. Two recent research threads address this from opposite ends. From the symbolic side, work on type-safe attention (Anonymous, 2026) gates attention by a domain ontology—an *olog*, in the sense of Spivak and Kent (2012)—and constrains generation to outputs that admit a proof of admissibility. From the neural side, work on compiled transformers (Lindner et al., 2023; Pérez et al., 2021) compiles deterministic programs directly into transformer weights, so one forward pass executes one program step.

These threads look unrelated—one is about typed knowledge graphs, the other about machine code. Both compile a discrete, typed transition system into transformer mechanics

© 2026 M.H. Lee.

so the forward pass implements the system's semantics, but each line states the pattern in its own vocabulary. We propose stating it once.

We take that object to be the *typed labeled transition system* (TLTS). Compiling a TLTS into a transformer is a functor from its path category into a category of compiled-transformer specifications: each legal path of state transitions maps to a forward-pass behavior, and gluing paths end-to-end corresponds to running the computation in sequence (composition is preserved). The paper develops the framework, identifies three distinct architectural loci where the transition relation can be enforced, and validates the resulting predictions empirically. Our contributions:

1. A categorical formalization of TLTS-compilation as a functor from the path category of a TLTS to a category of compiled transformer behaviors (Section 2).

2. A subsumption: procedural (program-compilation) and ontological (Olog-compilation) constructions are the deterministic (functional-δ) and nondeterministic (relational-δ) specializations of the same recipe (Section 4).

3. Three enforcement loci (in-FFN, pre-decoder, post-hoc) with different cost regimes (Section 3).

4. A prediction we confirm empirically: attention-layer reachability masking is insufficient for trajectory soundness; the decoder must also check direct-edge admissibility (Section 5).

5. A post-hoc verification protocol with an audit-certificate reference implementation that admits independent re-checking without model internals (Section 6).

2. TLTS and the Path Category

What we need from the math. The transformer is supposed to follow a domain rulebook: a finite, declared list of state shapes, allowed moves between them, and the labels those moves carry. The smallest object that captures all three is a labeled transition system; adding *types* to the states lets the rulebook constrain by state shape rather than by state identity. The rest of this section attaches one such object to a transformer's forward pass, so that one forward step on a token corresponds to one move in the rulebook. The properties we care about downstream—soundness (Section 2), enforcement loci (Section 3), and verifiability (Section 6)—are read off this correspondence.

A **typed labeled transition system** (TLTS) is a tuple $M = (T, L, \delta, t_0)$ where T is a set of types (state-shape classes), L is a set of labels (typed tokens), $\delta \subseteq T \times L \times T$ is a typed transition relation, and $t_0 \in T$ is the initial type. We write $t_1 \xrightarrow{\ell} t_2$ for $(t_1, \ell, t_2) \in \delta$. M is *functional* if δ is a partial function $T \times L \rightharpoonup T$; it is *relational* otherwise. We have $\mathsf{TLTS}_{\text{func}} \subsetneq \mathsf{TLTS}_{\text{rel}}$.

Path category. Every TLTS M induces a category $\mathcal{C}_M$ whose objects are types in T and whose morphisms $t_1 \to t_2$ are finite label sequences $(\ell_1, \ldots, \ell_n)$ realizing some trajectory in δ from t_1 to t_2. Composition is concatenation; identities are empty sequences. This is the free category on the directed multigraph of δ. When M is functional, $\mathcal{C}_M$ is *thin* (at

most one morphism per ordered pair); when M is relational, parallel morphisms exist. For a domain Olog $\mathcal{O}$ (Spivak, 2014), $\mathcal{C}_{\mathcal{O}}$ is the classifying category in the standard sense.

Compiled-transformer specifications. Fix a transformer architecture (layers, heads, hidden dimension d). A *compiled-transformer specification* is a triple (E, A, Φ) where $E : T \to \mathbb{R}^d$ is a type embedding (images separated under the residual-stream metric), $A \in \{0,1\}^{T \times T}$ is an attention admissibility mask, and Φ is a per-label gate-and-transition assignment (g_ℓ, τ_ℓ) realized in FFN rows. Specifications form a category CompT under embedding-preserving simulation.

The compilation functor. A *TLTS-compilation* of M is a functor $F_M : \mathcal{C}_M \to \mathsf{CompT}$ assembled from a recipe: choose E injective on T; set $A(t_q, t_k) = 1$ iff t_k is reachable from t_q in δ; realize δ at one or more of the three enforcement loci of Section 3. We say a compiled transformer C *realizes* F_M if its forward pass on label sequence $\ell_1, \ldots, \ell_n$ produces residuals $E(t_0), E(t_1), \ldots, E(t_n)$ with $(t_{i-1}, \ell_i, t_i) \in \delta$ at each step.

Proposition 1 (Trajectory-soundness).
Let C realize F_M with E injective and per-label gates g_ℓ sharp. Then for every input token sequence on which C produces a residual trace, that trace decodes to a valid trajectory of M.

The proof is direct: the attention mask A blocks information flow that violates type reachability; gate sharpness ensures exclusive ℓ-row activation; injectivity of E guarantees unique type recovery. Composition along the forward pass thus traces an admissible path in δ.

3. Three Enforcement Loci

Φ is left open because the transition relation can be enforced at three points in the inference pipeline. The three are not exclusive; production systems combine them. Costs differ:

Locus	Where δ checked	Per-step cost	Architectural cost
In-FFN	forward pass; FFN row gates	$O(1)$ per row	FFN re-compile
Pre-decoder	sampling; logit mask	$O(\|L\|)$ mask	none (drop-in)
Post-hoc	after gen.; trace audit	$O(n)$ for length-n	none on model

Table 1: Three loci for δ-enforcement.

The **in-FFN** locus is what compiled-transformer constructions (Lindner et al., 2023) use; gates *are* the enforcement. It is the strictest: the model cannot violate δ regardless of input, at the cost of constructing or fine-tuning weights for the specific TLTS.

The **pre-decoder** locus is the constrained-decoding setting (Willard and Louf, 2023; Geng et al., 2023). At each step, the admissible label set for the current type-state is computed from δ and logits over L are masked accordingly. The model is unchanged; enforcement happens at the sampling boundary.

The **post-hoc** locus does not enforce δ but audits that it was respected. Used alone, it implies a verify-and-retry loop that wastes compute; used in combination with one of

the first two loci, the verifier becomes a monitoring artifact rather than a flow-control gate (Section 6).

4. Subsumption: Procedural is Functional-δ Compilation

Procedural and ontological transformer constructions instantiate the same compilation recipe with different transition systems. A deterministic virtual machine has T = machine-state shapes, L = opcodes, δ = deterministic operational semantics. Compiled-transformer constructions (Lindner et al., 2023) bake δ into FFN gates (the in-FFN locus); a deterministic attention-key trick—in 2D embedding $(\text{step}, \text{value})$ with query direction $[1, 0]$, the most-recent-write is recovered as the argmax of a linear objective, which lies on the convex hull of the key set, giving $O(\log h)$ retrieval with hull-based indexing—substitutes for soft attention.

An Olog $\mathcal{O}$ is a TLTS with T = Olog types, L = morphism labels, δ = Olog edges plus equational identifications. The ontological attention layer used here (Anonymous, 2026) realizes A via reachability masking over $\mathcal{C}_{\mathcal{O}}$, with the categorical substrate from Spivak and Kent (2012). Proof-guided generation (Anonymous, 2026) realizes Φ via the pre-decoder locus: the proof object dictates the admissible label set per step, and a constrained decoder masks logits.

Proposition 2 (Subsumption, informal).
Procedural compilation $\subseteq \mathsf{Comp}(M)$ for $M \in \mathsf{TLTS}_{\text{func}}$. Ontological compilation $\subseteq \mathsf{Comp}(M)$ for $M \in \mathsf{TLTS}_{\text{rel}}$. Procedural compilation is a strict specialization of ontological compilation.

The hull-indexing trick of procedural compilation works *because* δ is functional: when the answer is unique, geometric indexing substitutes for soft attention. It does not lift to the relational case in general but does lift to functional sub-fragments of an Olog. This motivates a hybrid compilation strategy: in-FFN compile the deterministic sub-Olog, pre-decoder mask the relational remainder.

5. Experiments

What we are testing. The framework makes architectural predictions about constraint mechanics: that locus (B′) is incomplete, that (C) saves latency on functional types, that the cost of enforcement scales with prior–rule misalignment. These are claims about the constraint geometry, not about whether a particular trained LM happens to obey it. A synthetic prior holds model quality fixed and varies only the constraint policy, so the resulting numbers measure the mechanism. We return to a production attention layer in Section 5.4 to check that the synthetic-(B′) projection is a fair proxy for what a real ontological mask does.

The harness simulates an unconstrained model as a hand-crafted distribution $p_M(\ell \mid t)$; variants apply different enforcement loci as sampling policies.[1]

1. Code and full results in supplementary. Variant (B) attention-layer masking is simulated by a sampling-time projection (B') admitting any label whose target is reachable from the current state; the projection is faithful to the production attention mask, validated in Section 5.4.

Setup. We use a 7-type e-commerce Olog (Customer, Cart, Item, Checkout, Payment, Order, Delivery) with one branching node (Cart) and two terminals. We test two priors: Good (80% mass on admissible labels per type) and Bad (70% mass on a random favorite label per type, often inadmissible). Variants: (A) standard sample from p_M; (B′) sample from p_M masked by type-reachability of label-target; (C) deterministic step on functional types, (D) elsewhere; (D) sample from p_M masked by direct-edge admissibility from the current state.

5.1. Soundness, fluency, latency

Prior	Variant	Soundness	$\log P$/traj	Mean len	KL/step	μs/step
Good	A	48.1%	−2.54	2.57	0.22	11.0
Good	B′	63.3%†	−2.20	2.74	0.22	10.2
Good	C	100.0%	−1.48	3.51	0.22	6.0
Good	D	100.0%	−1.47	3.48	0.22	8.5
Bad	A	4.2%	−1.95	1.73	1.12	7.9
Bad	B′	4.5%†	−1.92	1.75	1.13	10.2
Bad	C	100.0%	−7.42	3.51	1.92	5.5
Bad	D	100.0%	−7.33	3.48	1.91	8.8

Table 2: Variant comparison on the e-commerce TLTS. N=1000 trajectories, max length 12, seed 42. Soundness is the fraction of trajectories whose every step is in δ. †Corrected (erratum): mean over 10 numpy seeds with sorted label sets (Good range 60.8–65.1%, Bad 3.3–5.6%); the other (B′) columns are the original single draw.

Three findings (Table 2). First, (C) and (D) reach 100% soundness regardless of prior alignment; (A)'s soundness tracks prior quality. Second, the fluency cost of enforcement scales with prior–δ misalignment: under Good, $\log P$ barely shifts; under Bad, trajectories are ≈ 5.5 nats less likely under the model's own distribution. KL-per-step is a cheap runtime diagnostic of this cost. Third, (C) is $\approx 30\%$ faster per step because deterministic steps on functional types (4 of 5 non-terminals here) skip the sampling pipeline.

5.2. Reachability is not admissibility

(B′) is barely better than (A) on soundness in Table 2: 63.3% vs. 48.1% under Good, 4.5% vs. 4.2% under Bad (corrected; see the erratum). Reachability admits the *destination* (type t' targeted by some label) without requiring the *step-edge* to exist as a δ-triple from the current state. The trajectory then breaks at the next step.

This is sharper at the limit. On a cyclic variant of the TLTS (adding Delivery → Customer), reachability becomes 100% universal across type pairs and (B′) collapses:

Implication. Reachability masking at the attention layer does not suffice; the decoder must also enforce direct-edge admissibility. Production systems that enforce δ only at the attention layer are unsound.

Topology	Reachability	(B′) snd, Good	(B′) snd, Bad
Acyclic	34.7%	63.3%†	4.5%†
Cyclic	100.0%	6.6%†	0.0%

Table 3: (B′) collapses under universal reachability; (C)/(D) remain 100% on both topologies (omitted). †Corrected (erratum): mean over 10 numpy seeds with sorted label sets.

5.3. Topology sweep: where the (C)/(D) tradeoff crosses

We sweep a parameterized TLTS family (length-5 chain plus $k \in \{0, \ldots, 5\}$ optional side-branches). The functional ratio is $(5 - k)/5$. The (C)/(D) latency ratio crosses 1.0 at functional ratio ≈ 0.2:

k	fn-ratio	(C)/(D) latency	(B′) snd, Good
0	1.00	**0.39**	54.0%
1	0.80	0.67	62.3%
2	0.60	0.80	62.2%
3	0.40	0.91	61.8%
4	0.20	0.99	60.1%
5	0.00	1.19	73.8%

Table 4: Topology sweep. (C)/(D) crossover at functional ratio ≈ 0.2; (B′) never approaches 100% regardless of topology.

Below the crossover, (C)'s deterministic-step shortcut on functional types pays for its bookkeeping overhead; above it, the overhead dominates. **Deployment heuristic**: use (C) when the functional fragment exceeds $\approx 20\%$ of the non-terminal type set; otherwise use (D).

5.4. Validation against production code

We bridge the cyclic TLTS into an OlogGraph and run the production ontological attention layer's mask construction on a 6-token trajectory window. The mask admits 6 self-pairs, 6 δ-direct-edge pairs, and **24 reachable-only pairs**—a 4:1 leakage ratio. With random-init weights, forward-pass attention mass distributes roughly uniformly across admitted pairs, landing 66.7% on reachable-only and only 16.4% on δ-direct-edge pairs. The (B′) projection matches what the production layer does. The production mask does not push mass toward direct-edge pairs; soundness must come from the decoder.

6. Verification Protocol and Audit Certificates

A TLTS-compiled transformer admits a four-step post-hoc verifier (Anonymous, 2026). Given the input, the forward-pass trace $h_0, \ldots, h_n$, and the declared TLTS:

1. **Decode.** Apply E^{-1} (or nearest-neighbor) to each h_i to recover candidate type $\hat{t}_i$; reject if any residual falls outside the codomain of E by more than ϵ.
2. **Recover labels** from the token-emission log.
3. **Check transitions.** For each i, verify $(\hat{t}_{i-1}, \ell_i, \hat{t}_i) \in \delta$.
4. **Check mask coherence.** Inspect attention-weight matrices: confirm no mass exceeds threshold on (q, k) pairs with $A(\hat{t}_q, \hat{t}_k) = 0$.

Upstream gating changes the verifier's role. Alone, the verifier is flow-control: failed generations retry and some inputs hang. Downstream of in-FFN or pre-decoder enforcement, it should never fail except on numerical drift, becoming a monitor and a public audit certificate.

Audit certificate. We ship a reference implementation that emits a JSON certificate per trajectory containing: a fingerprint (SHA-256-prefix over sorted T, L, δ) for drift detection; per-step records $(t_{\text{in}}, \ell, t_{\text{out}})$ plus diagnostics (in-δ flag, prior-argmax, forced flag, masked-KL); and a summary. A separate verifier consumes the JSON and the TLTS, returning pass/fail with reasons. It needs no model weights, no PyTorch, and no GPU. The reference verifier passes adversarial tests: tampering with a state-out (e.g., to a type $\notin T$) is caught by the per-step δ lookup; TLTS drift is caught by both fingerprint mismatch and per-step failures.

We recommend that compiled-transformer papers ship the JSON certificate alongside outputs and the TLTS spec alongside weights, with a verifier that runs without the model. Without this, compiled-transformer claims cannot be independently checked.

7. Related Work

Neurosymbolic AI. This work sits in the third wave of neurosymbolic integration (Garcez and Lamb, 2023; De Raedt et al., 2020), which integrates structure rather than couples modules. DeepProbLog (Manhaeve et al., 2018) and Logic Tensor Networks (Badreddine et al., 2022) align learned representations with logical structure via the loss function; we work at the architectural level (attention masks, FFN gates).

Compiled transformers. Tracr (Lindner et al., 2023) compiles RASP programs into transformers; ALTA generalizes the target. Theoretical work establishes Turing-completeness (Pérez et al., 2021; Bhattamishra et al., 2020). We treat these constructions as the functional-δ case of a broader pattern.

Type-safe attention and constrained decoding. The olog formalism (Spivak and Kent, 2012; Spivak, 2014) provides the categorical substrate; the type-safe attention layer built on it is reference code in our supplementary (Anonymous, 2026). Constrained decoding (Willard and Louf, 2023; Geng et al., 2023) is the grammar case of pre-decoder enforcement; we extend it to typed transition systems.

Verification. Mechanistic interpretability (Conmy et al., 2023) inspects circuits in trained models; we inspect forward passes against a declared compilation specification, which is a different problem with different failure modes.

8. Discussion

Over-constraint and the precision–fluency tradeoff. Constraint enforcement costs fluency: tight δ buys soundness at the price of likelihood. Known failure modes (mass-redistribution drift, premature lock-in, confidence collapse, dead-end states, topic-drift suppression) appear when p_M and δ disagree. Mitigations, in increasing order of effort: enrich the Olog (most over-constraint is an under-specified ontology), beam search with constraints, multi-proof rerank, hierarchical staging, backoff and abstention, and constraint-aware fine-tuning. Soft constraints are tempting but forfeit the soundness guarantee.

Model-size implications. Standard transformers devote parameters to suppressing implausible domain compositions: the distributional regularities of what does not happen in context. TLTS-compilation moves that knowledge into the Olog and frees model capacity for fluency and within-admissible-set selection. We predict a shift in domain-specific deployment from weight-heavy with a thin ontology to weight-light with a rich ontology, while open-domain models still need capacity. We do not test this here; constraint-aware fine-tuning is future work.

Limitations. The functor account is informal; promoting Proposition 1 to a theorem requires fixing residual-stream geometry and gate-sharpness precision. The empirical study uses synthetic priors; trained-model experiments are future work. Verification scales as $|T|^2|L|$, requiring lazy evaluation for very large ontologies.

9. Conclusion

TLTS-compilation unifies two previously separate threads, type-safe attention and procedural transformer compilation. It yields hybrid architectures, a post-hoc verification protocol, and a JSON-based audit certificate. All three are buildable today. We test the main empirical prediction next: that domain-specific deployment can substitute ontology engineering for raw scale.

References

Anonymous. Reference codebase, withheld for double-blind review, 2026. Code, data, and full results available in supplementary materials.

Samy Badreddine, Artur d'Avila Garcez, Luciano Serafini, and Michael Spranger. Logic tensor networks. *Artificial Intelligence*, 2022.

Satwik Bhattamishra, Arkil Patel, and Navin Goyal. On the computational power of transformers and its implications in sequence modeling. In *Conference on Computational Natural Language Learning (CoNLL)*, 2020.

Arthur Conmy, Augustine Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. In *Advances in Neural Information Processing Systems*, 2023.

Luc De Raedt, Robin Manhaeve, Sebastijan Dumancic, Thomas Demeester, and Angelika Kimmig. From statistical relational to neurosymbolic artificial intelligence. *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 2020.

Artur d'Avila Garcez and Luis C. Lamb. Neurosymbolic AI: The 3rd wave. *Artificial Intelligence Review*, 2023.

Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

David Lindner, János Kramár, Sebastian Farquhar, Matthew Rahtz, Thomas McGrath, and Vladimir Mikulik. Tracr: Compiled transformers as a laboratory for interpretability. In *Advances in Neural Information Processing Systems*, 2023.

Robin Manhaeve, Sebastijan Dumancic, Angelika Kimmig, Thomas Demeester, and Luc De Raedt. DeepProbLog: Neural probabilistic logic programming. In *Advances in Neural Information Processing Systems*, 2018.

Jorge Pérez, Pablo Barceló, and Javier Marinković. Attention is Turing-complete. In *Journal of Machine Learning Research*, 2021.

David I. Spivak. *Category Theory for the Sciences*. MIT Press, 2014.

David I. Spivak and Robert E. Kent. Ologs: A categorical framework for knowledge representation. *PLOS ONE*, 7(1):e24274, 2012.

Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.